Conclusion

Conclusion I

`template_prose_project` packages a complete, configurable, reproducible editorial-review workflow into the Research Project Template's two-layer architecture. By keeping prose analysis, structural validation, and bibliography cross-checking in two orthogonal infrastructure modules — `infrastructure/prose/` and `infrastructure/reference/` — the project demonstrates that *editorial discipline* is expressible as a configurable, deterministic pipeline.

Conclusion I

This exemplar follows a single house style:

Conclusion I

- `manuscript/config.yaml` is the only place run policy lives.
- `src/` is deliberately `infrastructure`-free: the report Protocols and the `parse_bib_keys`/`render_outline` helpers in `src/prose_facade.py` are the decoupling seam.
- Scripts in `scripts/` do only filesystem I/O, CLI argument handling, and the `infrastructure/` calls (e.g. `infrastructure.prose.analyze_manuscript`) on `src/`'s behalf.
- Every artefact in `output/` is regeneratable; `manuscript/references.bib` is curated and validated read-only by this project.

Conclusion I

The contribution of this exemplar is architectural: a *generic*, *reusable* prose-quality module that any project in the template can opt into, and a *minimal*, *configurable* exemplar wiring it to the bibliography and the manuscript pipeline.

Three concrete extensions follow naturally:

Conclusion I

1. **Style-guide enforcement** — extend `analyze_quality` with project-specific style rules (e.g. forbidden phrases, required terminology) read from `config.yaml`.
2. **Diff-aware review** — restrict the report to files modified since a given git ref so editorial review can run on every PR.
3. **LLM-assisted rewriting** — pipe long sentences and passive candidates into `infrastructure.llm` for suggested rewrites, using the same `seed=42, temperature=0.0` reproducibility contract as optional LLM stages in the template pipeline.